

Abstract of the PhD Dissertation: *Deep Inference for Graphical Theorem Proving*

Pablo Donato

1 Introduction

Proof assistants are software tools used to rigorously verify formal reasoning. Contemporary systems such as *Rocq* [57], *Lean* [41], and *Isabelle* [43] offer powerful frameworks for constructing formal proofs: they have been used successfully in various applications, ranging from the verification of advanced theorems in mathematics [28, 31, 16] to the certification of complex software such as programming language compilers [37], operating systems kernels [36] and cryptographic protocols [4, 59]. Yet they remain notoriously difficult to learn and use, requiring familiarity with text-based and command-line interfaces as well as a solid background in mathematical logic and functional programming. This usability barrier affects mainly newcomers, but also to some extent experienced users working in complex proof settings, limiting the application potential of proof assistants in two respects:

1. their use beyond the sole purpose of verifying correctness of proofs, for instance as expressive programming languages or as a medium for understanding and exploring mathematical concepts interactively;
2. hence their widespread adoption outside of academic research in industry and education.

In this dissertation, we are concerned primarily with the *human*-centered aspects (or “front-end”) of proof assistants: we aim to provide smoother means for the user to communicate her intent to the proof assistant, and conversely for the proof assistant to communicate its results and suggestions on how to solve problems. Furthermore, we restrict the scope of our investigations to the purely *logical* aspects at the heart of proof assistants — that is, the exact representation and manipulation of logical statements for the purpose of deductive reasoning.

The core thesis is that these challenges are not inherent to formal logic itself, but stem in great part from the *textual* mode of representation and interaction imposed by current tools, from the imperative (tactic-based) languages of Rocq and Lean — or their more declarative counterparts in Isabelle — to the foundational symbolic formalisms of proof theory and type theory that those languages build upon.

To address this, we introduce a novel paradigm called *Proof-by-Action* (PbA), in which users construct proofs through familiar graphical actions — such as pointing, clicking, dragging and dropping — performed directly on the logical content. Inspired by the *direct manipulation* principle from human-computer interaction [53], as well as earlier techniques like *Proof-by-Pointing* (PbP) [6] and *Proof-by-Linking* (PbL) [15], PbA seeks to bridge the gap between the user’s intent and the underlying formal system. Each user interaction is interpreted as a meaningful proof step, and the challenge is to ensure that these actions are semantically grounded, precise, and general enough to express a wide range of logical reasoning in a concise way.

To provide this foundation, we draw on the framework of *deep inference*, a modern approach to proof theory that enables fine-grained, local transformations within proofs. Deep inference was first introduced under this name by Guglielmi in the context of his *calculus of structures* [30], and it offers the flexibility and structural expressiveness needed to model direct manipulation in a formally sound way.

Chapter 1 of the dissertation introduces informally the necessary conceptual and historical background from the *proof theory* and *interactive theorem proving* literature, before presenting the motivations, research goals and methodology summarized above. It then goes on to outline the structure and main contributions of the thesis, as we are about to do in some more details in the next sections. To give a brief overview, the thesis is divided into two parts:

The first part formalizes PbA in the traditional setting of *symbolic* logic. It introduces *Actema*, a graphical user interface (GUI) where gestural actions upon formulas perform logical inferences [21] (Chapter 2), and defines a semantic framework specifically for drag-and-drop actions based on the *subformula linking* technique in deep inference [13] (Chapter 3). It also explores more advanced features like inductive reasoning and simplification (Chapter 4), extends PbA to classical logic via multi-conclusion sequents (Chapter 5), and integrates the interface into Rocq via the *coq-actema* plugin [20], translating GUI actions into verified proof terms (Chapter 6).

The second part investigates *iconic* systems for logic, where goals are represented as diagrams rather than formulas. It introduces *bubble calculi* which extend the nested sequents of Brünnler [10] with topological structure (Chapters 7–8), gives a modern analysis of C. S. Peirce’s diagrammatic proof system of *existential graphs* [50] (Chapter 9), and revisits the latter in an intuitionistic light through the design of the *flower calculus* [18] (Chapter 10). The flower calculus is the foundation for the *Flower Prover*, another prototype of GUI for interactive proof building designed to work well with modern devices based on touch interactions [17].

Publications Most of the content of Chapters 2 and 3 has been previously published in [19]. A shortened version of Chapters 9 and 10 has been published in [18]. All other chapters present completely original and personal work.

In sum, the thesis contributes both a theoretical framework and a set of practical tools for reimagining how users interact with formal proofs. By treating direct manipulation as a first-class principle of proof theory, it opens the way for new kinds of logical literacy that are more intuitive, visual, and expressive.

2 Proof-by-Action

This chapter explains how both *click* and *drag-and-drop* (DnD) actions upon the formulas of a sequent can implement proof construction operations corresponding to the core logic; that is, how they deal with logical connectives, quantifiers, and equality. We present these core principles through various illustrations and examples in first-order logic.

This chapter focuses mostly on two broad categories of DnD actions: *forward steps*, where two existing facts are combined to deduce a new one, and *backward steps*, where a known fact is used to transform the goal. These operations are backed by a proof-theoretic technique known as *subformula linking* that will be formally defined in the next chapter. It can roughly be understood as a kind of operational semantics, where the result of a DnD action is computed by applying rewriting rules inside logical formulas. This is illustrated in examples by giving both the resulting

new statement and its derivation computed by the system. The set of rewriting rules is also given at the end of the chapter for reference.

We have started to implement the paradigm in a prototype named *Actema* (for “**A**ctive **m**athematics”) running through a web HTML/JavaScript interface. At the time of writing, a standalone version of the prototype (i.e., which does not use an existing theorem prover as its backend) is publicly available online [21]. This possibility to experiment in practice, even though yet on a small scale, gave valuable feedback for crafting the way DnD actions are to be translated into proof construction steps in an intuitive and practical way.

Overall, this chapter serves as a motivational entry point into the thesis, demonstrating how logical reasoning can be made more tangible through carefully designed graphical interactions that remain formally sound.

3 Subformula Linking

In this chapter we engage in a thorough analysis of the logical semantics of DnD actions, which were introduced informally through examples in Chapter 2. We do this mainly from the formal perspective of deep inference proof theory, following the original work of K. Chaudhuri on *subformula linking* [13]. But we always keep in mind the intended application to proof assistants, by motivating various design choices — actual or prospective — as ways to improve the *user experience* (*UX*) of interactive proof building.

3.1 Linkages This section introduces the notion of a *linkage* as the formal representation of a DnD action connecting two selected subformulas. It defines the standard deep inference notions of *context* and *polarity*, and details how linkages are structured to determine the kind of interaction intended by the user.

3.2 Validity Here, a precise criterion is established for when a linkage is considered valid, combining two key conditions: opposite polarities between the linked subterms and their unifiability. These conditions ensure that the interaction leads to productive simplification rather than vacuous transformation. This is a novelty and the main point of divergence with the original approach of Chaudhuri.

3.3 Describing DnD Actions This section outlines the full process by which a DnD action is transformed into a valid proof step, including the generation, validation, and execution of linkages. It defines a deterministic and terminating procedure that guarantees logical soundness by alignment with the system’s rewriting rules.

3.4 Soundness This part formally proves that all rewriting steps preserve logical validity (Theorem 3.4.1). It establishes contextual soundness (Lemma 3.4.4) by ensuring that every transformation corresponds to a provable sequent in intuitionistic first-order logic with equality.

3.5 Productivity The section shows that for every valid linkage, the system can always reach a final interaction point (corresponding to an identity axiom or substitution rule for equality) in a finite number of steps (Theorem 3.5.2). This ensures that DnD actions are not just sound but also effective in progressing toward the completion of proofs.

3.6 Focusing The set of rewriting rules for subformula linking turns out to be non-confluent. To resolve ambiguities in rule application, this section adapts the focusing discipline well known from the sequent calculus literature, preferring invertible rules to guide the rewriting strategy. This helps

avoid dead ends and heuristically makes the semantics of DnD actions more natural (at least in common situations).

3.7 Completeness This section demonstrates that the set of DnD actions, taken alone, can form a complete deductive system for first-order intuitionistic logic. It constructs a simulation argument to relate the deep inference system to standard sequent calculi (Theorem 3.7.1).

3.8 Related Works The final section surveys related literature, comparing this approach to subformula linking with alternatives in deep inference [14], proof nets [27, 56, 34], automated theorem proving [2], program verification [42] and functional programming [23]. It highlights how the notion of valid linkage simplifies usability while retaining semantic power.

4 Proof-by-Action in Practice

In the previous chapters, we explained the core principles of our so-called Proof-by-Action paradigm and especially of its drag-and-drop actions, first through basic and abstract examples in Chapter 2, and then from a proof-theoretical perspective in Chapter 3. The goal of this chapter is to provide a better sense of what proofs by action look like in practice, and how they compare to more traditional approaches to interactive theorem proving. To that effect, we perform a case study of a few select examples, unrolling and commenting in detail one or many of their proofs. Although still basic, they are fully fledged, concrete logical riddles or mathematical problems that one might give as exercises to an undergraduate student learning formal proofs. Note that our analysis stays quite informal and opinionated: a more systematic approach such as a user study would allow for a better evaluation of our paradigm, but at the time of writing of this thesis the Actema prototype was not mature enough to conduct a project of this scale.

4.1 Forward Reasoning This section presents a proof of a simple logical riddle taken from the Edukera project [51] within the Actema interface. The direct manipulation approach reduces the cognitive overhead of naming hypotheses and structuring tactic sequences, especially for beginners. A comparison with the equivalent Rocq script shows that the graphical version is not only more accessible but often faster to construct.

4.2 Sets and Functions Here, the paradigm is extended to reasoning about functions and sets. We distinguish between nominal, axiomatic, and computational *definitions*, each supported by corresponding contextual menu actions in Actema such as **Simplify** or **Unfold**. The case study demonstrates that PbA can accommodate both explicit and implicit styles of proof found in various proof assistants.

4.3 Peano Arithmetic The final section tackles a more complex proof: the commutativity of addition on natural numbers, using *induction*. It shows how inductive steps can be triggered via contextual actions, and how automation (e.g. simplification and lemma search) can be performed with more power, precision and ease than is possible with a textual language. The close match between the graphical proof and a Rocq script suggests promising directions for compiling PbA proofs to traditional languages.

5 Parallel Conclusions and Classical Logic

This chapter extends the Proof-by-Action paradigm to support sequents with multiple conclusions, as used by Gentzen for classical logic [26]. It explores how graphical reasoning can be adapted

to this setting by introducing a new form of subformula linking — *parallel interaction* — and formalizes it through an original set of rules and linking semantics.

5.1 Backtracking Discusses the limitations of single-conclusion systems in handling disjunctive goals and the cognitive burden of backtracking in traditional proof assistants. The section motivates the need for invertible rules and the expressive power of multi-conclusion sequents for reducing backtracking, as noticed by other authors [3].

5.2 Implementation in Theorem Provers Notes that despite the advantages, most modern proof assistants avoid multi-conclusion sequents due to UI complexity. The Actema interface circumvents these limitations through direct manipulation, making multiple conclusions manageable.

5.3 Click Actions Explains how click actions in Actema are extended to support multi-conclusion sequents. Modifications to disjunction and implication actions allow Actema to capture both intuitionistic and classical behavior, depending on how extra conclusions are treated.

5.4 Drag-and-Drop Actions Introduces the concept of *parallel linking* between two conclusions, interpreted as disjunction. This gives rise to a third mode of interaction — parallel reasoning — complementing forward and backward reasoning, and supported by a new interaction operator.

5.5 Metatheory of Parallel Reasoning Establishes the soundness of parallel rules in the classical setting, while noting their incompatibility with intuitionistic logic. Updates are made to the validity conditions for linkages to allow for parallel interactions, and extensions to the core metatheory of Chapter 3 are proposed to accommodate classical completeness and productivity.

6 Integration in a Proof Assistant

This chapter describes how the Proof-by-Action paradigm has been integrated into the Rocq proof assistant through the `coq-actema` system [20]. The goal is to connect the graphical interface developed in earlier chapters with an industrial-strength proof assistant, enabling users to benefit from its large ecosystem of existing tools and formalizations. This is also a necessary step to evaluate to which extent the PbA paradigm scales to real-world mathematical developments.

6.1 Actema Introduces the architecture of the Actema web application [21], which separates a JavaScript frontend from an OCaml backend compiled via `js_of_ocaml` [58]. The two components communicate via an object-oriented API, enabling interaction between the UI and the proof engine.

6.2 Why a Plugin? Argues for integrating Actema via a dedicated Rocq plugin rather than modifying Rocq itself or relying on high-level protocols like SerAPI [25]. This design enables Actema to function as an alternative, graphical proof view within standard Rocq IDEs such as RocqIDE or Proof General.

6.3 The `coq-actema` System Describes the full-stack communication setup, where a plugin running inside Rocq interacts with the Actema frontend over HTTP. This enables real-time, bidirectional proof interaction and allows packaging the interface as a standalone Electron application for ease of deployment.

6.4 Interaction Protocol Details the communication protocol using UML-style sequence diagrams. These diagrams illustrate how proof goals are exported, actions are queried and selected, and proof steps are sent back to Rocq to be compiled as executable tactics.

6.5 Compiling Actions Explains how user actions in the graphical interface are serialized via ATD [39] and compiled into Rocq tactics through metaprogramming, using the $\mathcal{L}_{tac}$ metalanguage to implement the semantics of subformula linking in a deep embedding of first-order logic.

6.6 Future Work Outlines limitations of the current system, such as incomplete support for advanced Rocq features and lack of robust proof maintenance tools. It also addresses the issue of proof evolution and proposes directions for improving the long-term usability of Actema for real-world developments.

7 Asymmetric Bubble Calculus

This chapter introduces the *asymmetric bubble calculus* as a new kind of nested sequent system [10]. By internalizing the notion of subgoals as “bubbles”, it provides a more topological and hierarchical way to represent logical contexts. This enables efficient sharing of hypotheses between subgoals, as well as more local proof steps.

7.1 The chemical metaphor Explains how Actema’s interface can be interpreted through a *chemical* metaphor, with formulas viewed as reactants that can break or merge under user-driven proof actions, inspired by the *chemical abstract machine* (CHAM) of Berry and Boudol [5]. This perspective guides the design of *bubble* boundaries akin to the CHAM’s *membrane* mechanism, that manage context scope without relying on multiple tabs or separate subgoals.

7.2 Bubbles and solutions Introduces bubbles as a device for delineating subgoals and allowing selective flow of hypotheses. The idea of a “*solution*” generalizes a sequent by blending hypotheses, conclusions and subgoals into a single space, giving users a more direct handle on scoping.

7.3 Asymmetric calculus Presents the collection of local rewriting rules constituting the asymmetric bubble calculus, specialized for single-conclusion intuitionistic logic. Each rule concerns either structural manipulations (like context sharing) or logical connectives (like introductions and eliminations), and it is shown how these rules remain faithful to a standard sequent system.

7.4 Back to Proof-by-Action Connects the rules for manipulating bubbles to the gestural actions of PbA described in the first part of the thesis. Interaction with bubbles can be represented faithfully in a *zoomable user interface* [48] equipped with a rudimentary *physics* engine for collision detection, suggesting a continuous, topological model of scoping and subgoal management that is more intuitive and discoverable.

8 Symmetric Bubble Calculi

In this chapter, we explore to what extent the bubble calculus of Chapter 7 can be made more symmetric by relaxing the restriction that solutions must contain at most one conclusion. At a surface level, our approach is similar to that of Gentzen, who went from his single-conclusion sequent calculus LJ to the multi-conclusion calculus LK [26]. Like him, we uncover beautiful dualities that were hidden by the asymmetry of the initial calculus. But by sticking unwaveringly to intuitionism, we are led to the exotic territory of bi-intuitionistic logic, an intermediate logic that conservatively extends intuitionistic logic, and in particular does not prove the law of excluded middle.

An underlying thread of our investigation is the quest for a fully iconic proof system, where all logical connectives can be replaced by appropriate new kinds of bubbles. This makes us rediscover many principles already studied in the deep inference literature, with topological intuitions of the

bubble metaphor shedding a new light on them. We end up with two symmetric bubble calculi, each with its own tradeoff on the properties satisfied by inference rules. In particular, the ability of the calculi to factorize *backward* in addition to forward proof steps might prove useful to build concise proofs, all through direct manipulation.

The chapter is organized as follows: in Section 8.1 we motivate our quest for a system where all introduction rules for logical connectives are invertible, to reduce non-determinism in proof search and enable a fully iconic approach to proof building. To that effect, we relax in Section 8.2 the restriction to single-conclusion solutions, which requires a new distinction between saturated and unsaturated solutions. This gives rise in Section 8.3 to an extension of the syntax of solutions, where bubbles can themselves be polarized. In Section 8.4 we identify key properties that have guided the design of inference rules, some of which were already aimed for implicitly throughout the evolution of our concept of bubble.

In Section 8.5 we introduce a core symmetric bubble calculus for classical logic called system B, in reference to the symmetric system L of Herbelin [33]. Then in Section 8.6 we prove the soundness of system B with respect to various classes of algebras, showing that by removing selectively some inference rules that define the porosity of polarized bubbles (as summarized in Figure 8.12), one gets intuitionistic (Corollary 8.6.18), dual-intuitionistic (Corollary 8.6.21), and bi-intuitionistic logic (Corollary 8.6.18) as fragments. In Section 8.7 we support this claim by showing that the bi-intuitionistic fragment is not only sound, but also cut-free complete with respect to the cut-free nested sequent calculus DBiInt of Postniece [49] (Theorem 8.7.2).

Finally, in Section 8.8 we introduce a fully invertible variant of system B that we conjecture to be complete, and present a canonical way to search for proofs in this system. Unfortunately, invertibility does not entail the full iconicity of the system, and we reflect on the fundamental reasons that might prevent any variant of system B from being fully iconic.

9 Existential Graphs

C. S. Peirce is famous for his contributions to symbolic logic, including among others his eponymous law for classical logic, and his pioneering work on the algebra of relations and quantification [46]. But far less widespread are his achievements in the realm of diagrammatic logic, or *iconic* logic as Shin calls it [52]. He dedicated a large chunk of his life to the investigation of graphical systems, starting in 1882 with the *entitative graphs* and culminating with the *existential graphs* (EGs), which he developed from 1896 until his death in 1914 [50]. Interestingly, Peirce perceived existential graphs as his “chef d’oeuvre”, and that they “ought to be the logic of the future.”¹

Recent works have started to realize this vision: for example, Sowa based his conceptual graphs for computerized knowledge representation on EGs [55]; Brady, Trimble [8, 9], Gianluca, Rocco [12] and Haydon, Sobociński [32] proposed various reconstructions of EGs through the lens of topology and category theory; lastly, Melliès, Zeilberger [40] and Bonchi et al. [7] refined respectively the interpretations of [9] and [32] by making further connections with linear logic [27] and linear bicategories. The full story has yet to be told, but we hope that our work will constitute one more step towards the vision Peirce had in mind.

In this chapter, we propose a self-contained exposition of EGs, that tries at the same time to be faithful to the original presentation of the systems by Peirce, and more modern in some aspects of their formalization. The goal is to familiarize the reader with the unique approach to proofs inherent to EGs, which can be difficult to relate to more standard frameworks like Hilbert and

¹Both citations are sourced in [50, p. 11].

Gentzen proof systems, and even deep inference proof systems like the calculus of structures. This shall prove useful to get a good understanding of the historical and technical foundations behind our *flower calculus*, to be introduced in Chapter 10.

The chapter is organized as follows: we start in Section 9.1 by presenting the diagrammatic syntax of the system Alpha of EGs for classical propositional logic. In Section 9.2, we introduce the inference rules of Alpha for manipulating EGs, called *illative transformations* by Peirce. In Section 9.3, we give an equivalent formulation of the syntax and rules of Alpha as a multiset rewriting system. In Section 9.4, we formalize a variant of the (De)iteration principle described by Peirce in [47] that eliminates the need for the Double-cut principle, and discuss how it was motivated by Peirce’s quest for illative atomicity. In Section 9.5, we take advantage of our reformulation to give a simple proof of soundness for Alpha, based on a direct truth-evaluation of graphs. In Section 9.6, we give a syntactic proof of completeness for Alpha, by simulating the calculus of structures SKS of Brünnler and Tiu [11]. In this way, Alpha is shown to have subsystems that inherit the locality property of SKS, and where the Deletion and Insertion rules are respectively admissible for provability and refutability, making Alpha *analytic*. In Section 9.7, we illustrate the original mechanism of *lines of identity* used by Peirce to handle quantifiers and equality in his Beta system. We end in Section 9.8 by showing how to recast lines of identity in a more traditional binder-based syntax.

10 Flower Calculus

In this final chapter we introduce the *flower calculus*, a novel proof system for intuitionistic predicate logic based on syntactic objects called *flowers*. We start by explaining how flowers stem from considerations in graphical logic, and more specifically from an intuitionistic variant of the existential graphs of C. S. Peirce proposed by A. Oostra [45, 44, 60]. We then present our inductive syntax for flowers, reminiscent at the same time of the nested sequents of deep inference proof theory [10, 24, 29, 38], and the geometric/coherent formulas of categorical logic [54, 22, 35].

A salient feature of our calculus inherited from EGs, is that it is *fully iconic*: it dispenses completely with the traditional notion of symbolic formula, operating instead as a rewriting system on flowers containing only atomic predicates. We also propose a notion of proof geared towards analyticity results à la Gentzen, suggesting new rules absent from other works on intuitionistic EGs. This allows us to prove admissibility theorems for many rules, including Peirce’s deletion rule which is a variant of Gentzen’s cut rule. These results are obtained as a consequence of our soundness and completeness proofs with respect to Kripke semantics, in the spirit of the *normalization-by-evaluation* technique [1].

Furthermore, the kernel of rules targeted by completeness is fully *invertible*, a desirable property in both automated and interactive theorem proving. This is illustrated by our implementation of the Flower Prover [17], an early prototype of GUI for ITPs that uses the rules of the flower calculus both for direct manipulation of flowers in its frontend, and automated simplification of goals in its backend.

The chapter is organized as follows: in Section 10.1, we retrace the origin of Oostra’s syntax for intuitionistic existential graphs (IEGs) as a natural generalization of the *scroll*, an icon for implication introduced by Peirce that inspired the very creation of EGs. In Section 10.2, we explain how flowers are really just a fun and metaphorical way to draw IEGs, and proceed to give them an inductive, multiset-based syntax as in Section 9.3. In Section 10.3, we introduce the full

set of inference rules of the flower calculus as well as our notion of proof, and prove a few syntactic properties, including two deduction theorems (Theorems 10.3.2 and 10.3.4). In Section 10.4, we give a direct Kripke semantics to flowers, avoiding the need for translations to and from formulas. In Section 10.5, we show that the rules of the flower calculus are valid with respect to our Kripke semantics (Theorem 10.5.13), and in Section 10.6, we identify a complete fragment of the system where all rules are both *analytic* and *invertible* (Theorem 10.6.6). This entails the admissibility of all rules outside of this fragment, and as a consequence the analyticity of the system. We exploit these properties in Section 10.7 by describing an algorithm for fully automated proof search in the propositional fragment; unfortunately, the current version of the algorithm is neither terminating nor complete. Then in Section 10.8, we give an overview of the Flower Prover, a prototype of GUI in the Proof-by-Action paradigm whose actions map directly to the rules of the flower calculus, and which integrates nicely with (a restricted version of) our search procedure. We conclude in Section 10.9 with a comparison to some related works, and a discussion of future directions and applications we envision.

References

- [1] Andreas Abel. “Normalization by Evaluation: Dependent Types and Impredicativity”. Habilitationsschrift. Ludwig-Maximilians-University Munich, May 2013. URL: <http://www.cse.chalmers.se/~abela/habil.pdf>.
- [2] Andrews. “Refutations by Matings”. In: *IEEE Transactions on Computers* C-25.8 (1976), pp. 801–807.
- [3] Edward W. Ayers. “A Tool for Producing Verified, Explainable Proofs.” en. PhD thesis. University of Cambridge, 2021.
- [4] Gilles Barthe et al. “EasyCrypt: A Tutorial”. In: *Foundations of Security Analysis and Design VII: FOSAD 2012/2013 Tutorial Lectures*. Ed. by Alessandro Aldini, Javier Lopez, and Fabio Martinelli. Cham: Springer International Publishing, 2014, pp. 146–166. ISBN: 978-3-319-10082-1. DOI: 10.1007/978-3-319-10082-1_6. URL: https://doi.org/10.1007/978-3-319-10082-1_6.
- [5] Gerard Berry and Gerard Boudol. “The chemical abstract machine”. In: *Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. POPL ’90. New York, NY, USA: Association for Computing Machinery, Dec. 1989, pp. 81–94. ISBN: 978-0-89791-343-0. DOI: 10.1145/96709.96717. URL: <https://doi.org/10.1145/96709.96717> (visited on 05/02/2022).
- [6] Yves Bertot, Gilles Kahn, and Laurent Théry. “Proof by pointing”. In: *Theoretical Aspects of Computer Software*. Ed. by Masami Hagiya and John C. Mitchell. Red. by Gerhard Goos and Juris Hartmanis. Vol. 789. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 141–160. ISBN: 978-3-540-57887-1 978-3-540-48383-0. DOI: 10.1007/3-540-57887-0_94. URL: http://link.springer.com/10.1007/3-540-57887-0_94 (visited on 04/21/2020).
- [7] Filippo Bonchi et al. *Diagrammatic Algebra of First Order Logic*. arXiv:2401.07055 [cs, math]. Jan. 2024. DOI: 10.48550/arXiv.2401.07055. URL: <http://arxiv.org/abs/2401.07055> (visited on 01/21/2024).

- [8] Geraldine Brady and Todd H. Trimble. “A categorical interpretation of C.S. Peirce’s propositional logic Alpha”. en. In: *Journal of Pure and Applied Algebra* 149.3 (June 2000), pp. 213–239. ISSN: 00224049. DOI: 10.1016/S0022-4049(98)00179-0. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0022404998001790> (visited on 02/18/2024).
- [9] Geraldine Brady and Todd H. Trimble. *A String Diagram Calculus for Predicate Logic and C. S. Peirce’s System Beta*. June 2000. URL: <https://ncatlab.org/nlab/files/BradyTrimbleString.pdf> (visited on 02/22/2023).
- [10] Kai Brännler. “Deep sequent systems for modal logic”. en. In: *Archive for Mathematical Logic* 48.6 (July 2009), pp. 551–577. ISSN: 0933-5846, 1432-0665. DOI: 10.1007/s00153-009-0137-3. URL: <http://link.springer.com/10.1007/s00153-009-0137-3> (visited on 01/10/2024).
- [11] Kai Brännler and Alwen Fernanto Tiu. “A Local System for Classical Logic”. en. In: *Logic for Programming, Artificial Intelligence, and Reasoning*. Ed. by Robert Nieuwenhuis and Andrei Voronkov. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2001, pp. 347–361. ISBN: 978-3-540-45653-7. DOI: 10.1007/3-540-45653-8_24.
- [12] Gianluca Caterina and Rocco Gangle. “A New Syntax for Diagrammatic Logic: A Generic Figures Approach”. en. In: *Numerical Computations: Theory and Algorithms*. Ed. by Yaroslav D. Sergeyev and Dmitri E. Kvasov. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, pp. 43–58. ISBN: 978-3-030-40616-5. DOI: 10.1007/978-3-030-40616-5_4.
- [13] Kaustuv Chaudhuri. “Subformula Linking as an Interaction Method”. In: *Interactive Theorem Proving*. Ed. by Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie. Red. by David Hutchison et al. Vol. 7998. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 386–401. ISBN: 978-3-642-39633-5 978-3-642-39634-2. DOI: 10.1007/978-3-642-39634-2_28. URL: http://link.springer.com/10.1007/978-3-642-39634-2_28 (visited on 04/21/2020).
- [14] Kaustuv Chaudhuri. “Subformula Linking for Intuitionistic Logic with Application to Type Theory”. In: *Automated Deduction - CADE 28 - 28th International Conference on Automated Deduction, Virtual Event, July 12-15, 2021, Proceedings*. Ed. by André Platzer and Geoff Sutcliffe. Vol. 12699. Lecture Notes in Computer Science. Springer, 2021, pp. 200–216. DOI: 10.1007/978-3-030-79876-5_12. URL: https://doi.org/10.1007/978-3-030-79876-5_12.
- [15] Kaustuv Chaudhuri et al. *Certifying Proof-By-Linking*. en. Sept. 2022. URL: <https://inria.hal.science/hal-04317972> (visited on 12/09/2023).
- [16] Johan Commelin and Adam Topaz. *Abstraction Boundaries and Spec Driven Development in Pure Mathematics*. Sept. 2023. DOI: 10.48550/arXiv.2309.14870. arXiv: 2309.14870 [math]. (Visited on 04/11/2025).
- [17] Pablo Donato. *flower-prover*. Oct. 2023. URL: <https://github.com/Champitoad/flower-prover> (visited on 12/08/2023).
- [18] Pablo Donato. “The Flower Calculus”. In: *9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024)*. Ed. by Jakob Rehof. Vol. 299. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 5:1–5:24. ISBN: 978-3-95977-323-2. DOI: 10.4230/LIPIcs.FSCD.2024.5. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2024.5>.

- [19] Pablo Donato, Pierre-Yves Strub, and Benjamin Werner. “A drag-and-drop proof tactic”. In: *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2022. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 197–209. ISBN: 9781450391825. DOI: 10.1145/3497775.3503692. URL: <https://doi.org/10.1145/3497775.3503692>.
- [20] Pablo Donato and Benjamin Werner. *coq-actema*. 2023. URL: <https://github.com/Champitoad/coq-actema> (visited on 02/24/2024).
- [21] Pablo Donato et al. *The Actema prototype, standalone version*. <https://actema.xyz>. 2021.
- [22] Roy Dyckhoff and Sara Negri. “Geometrization of First-Order Logic”. In: *Bulletin of Symbolic Logic* 21.2 (2015), pp. 123–163. DOI: 10.1017/bsl.2015.7.
- [23] Conal M. Elliott. “Tangible Functional Programming”. In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming*. ICFP ’07. Freiburg, Germany: Association for Computing Machinery, 2007, pp. 59–70. ISBN: 9781595938152. DOI: 10.1145/1291151.1291163. URL: <https://doi.org/10.1145/1291151.1291163>.
- [24] Melvin Fitting. “Nested Sequents for Intuitionistic Logics”. en. In: *Notre Dame Journal of Formal Logic* 55.1 (2014), pp. 41–61. ISSN: 0029-4527. DOI: 10.1215/00294527-2377869. URL: <http://projecteuclid.org/euclid.ndjfl/1390246437> (visited on 04/30/2020).
- [25] Emilio Jesús Gallego Arias. “SerAPI: Machine-Friendly, Data-Centric Serialization for COQ”. working paper or preprint. Oct. 2016. URL: <https://hal-mines-paristech.archives-ouvertes.fr/hal-01384408>.
- [26] Gerhard Gentzen. “Untersuchungen über das logische Schließen. I”. de. In: *Mathematische Zeitschrift* 39.1 (Dec. 1935), pp. 176–210. ISSN: 1432-1823. DOI: 10.1007/BF01201353. URL: <https://doi.org/10.1007/BF01201353> (visited on 12/10/2023).
- [27] Jean-Yves Girard. “Linear logic”. In: *Theoretical Computer Science* 50.1 (1987), pp. 1–101. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(87\)90045-4](https://doi.org/10.1016/0304-3975(87)90045-4). URL: <http://www.sciencedirect.com/science/article/pii/0304397587900454>.
- [28] Georges Gonthier. “Formal Proof—The Four- Color Theorem”. In: 55.11 (2008).
- [29] Nicolas Guenot. “Nested Deduction in Logical Foundations for Computation”. en. PhD thesis. Ecole Polytechnique X, Apr. 2013. URL: <https://pastel.archives-ouvertes.fr/pastel-00929908> (visited on 03/05/2021).
- [30] Alessio Guglielmi. *A Calculus of Order and Interaction*. Tech. rep. Technische Universität Dresden, 1999. URL: https://www.researchgate.net/publication/2807151_A_Calculus_of_Order_and_Interaction.
- [31] Thomas Hales. *The Formal Proof of the Kepler Conjecture: A Critical Retrospective*. Feb. 2024. DOI: 10.48550/arXiv.2402.08032. arXiv: 2402.08032 [math]. (Visited on 04/11/2025).
- [32] Nathan Haydon and Paweł Sobociński. “Compositional Diagrammatic First-Order Logic”. en. In: *Diagrammatic Representation and Inference*. Ed. by Ahti-Veikko Pietarinen et al. Vol. 12169. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, pp. 402–418. ISBN: 978-3-030-54248-1 978-3-030-54249-8. DOI: 10.1007/978-3-030-54249-8_32. URL: http://link.springer.com/10.1007/978-3-030-54249-8_32 (visited on 02/05/2023).
- [33] Hugo Herbelin. *Duality of computation and sequent calculus : a few more remarks*. Online Notes. 2008. URL: <http://pauillac.inria.fr/~herbelin/publis/full-dual-1k.pdf>.

- [34] Dominic J. D. Hughes. *First-order proofs without syntax*. 2019. arXiv: 1906.11236 [math.LO]. URL: <https://arxiv.org/abs/1906.11236>.
- [35] Peter T. Johnstone. *Sketches of an Elephant: A Topos Theory Compendium*. Vol. 2. Oxford Logic Guides. Oxford, England: Clarendon Press, Sept. 2002.
- [36] Gerwin Klein et al. “seL4: Formal Verification of an OS Kernel”. In: *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*. SOSP ’09. New York, NY, USA: Association for Computing Machinery, Oct. 2009, pp. 207–220. ISBN: 978-1-60558-752-3. DOI: 10.1145/1629575.1629596. (Visited on 04/11/2025).
- [37] Xavier Leroy. “Formal Verification of a Realistic Compiler”. In: *Commun. ACM* 52.7 (July 2009), pp. 107–115. ISSN: 0001-0782. DOI: 10.1145/1538788.1538814. (Visited on 04/11/2025).
- [38] Tim S. Lyon. “Nested Sequents for Intuitionistic Modal Logics via Structural Refinement”. en. In: *Automated Reasoning with Analytic Tableaux and Related Methods*. Ed. by Anupam Das and Sara Negri. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2021, pp. 409–427. ISBN: 978-3-030-86059-2. DOI: 10.1007/978-3-030-86059-2_24.
- [39] Martin Jambon. *OCaml support – atdgen*. <https://atd.readthedocs.io/en/latest/atdgen.html>. 2023. (Visited on 07/05/2023).
- [40] Paul-André Melliès and Noam Zeilberger. “A bifibrational reconstruction of Lawvere’s presheaf hyperdoctrine”. In: *arXiv:1601.06098 [cs, math]* (Aug. 2016). arXiv: 1601.06098. URL: <http://arxiv.org/abs/1601.06098> (visited on 02/07/2022).
- [41] Leonardo de Moura and Sebastian Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction – CADE 28*. Ed. by André Platzer and Geoff Sutcliffe. Cham: Springer International Publishing, 2021, pp. 625–635. ISBN: 978-3-030-79876-5.
- [42] Ike Mulder and Robbert Krebbers. “Unification for Subformula Linking under Quantifiers”. In: *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2024. New York, NY, USA: Association for Computing Machinery, Jan. 2024, pp. 75–88. ISBN: 9798400704888. DOI: 10.1145/3636501.3636950. URL: <https://dl.acm.org/doi/10.1145/3636501.3636950> (visited on 01/21/2024).
- [43] Tobias Nipkow, Lawrence C Paulson, and Markus Wenzel. *Isabelle/HOL: a proof assistant for higher-order logic*. Vol. 2283. Springer Science & Business Media, 2002.
- [44] Arnold Oostra. *Gráficos existenciales Beta intuicionistas*. es. Cuadernos de Sistemática Peirceana 3. Centro de Sistemática Peirceana, 2011.
- [45] Arnold Oostra. *Los gráficos Alfa de Peirce aplicados a la lógica intuicionista*. es. Cuadernos de Sistemática Peirceana 2. Centro de Sistemática Peirceana, 2010.
- [46] Charles Sanders Peirce. “On the Algebra of Logic: A Contribution to the Philosophy of Notation [Continued in Vol. 7, no. 3]”. In: *American Journal of Mathematics* 7.2 (1885). Publisher: The Johns Hopkins University Press, pp. 180–196. ISSN: 0002-9327. DOI: 10.2307/2369451. URL: <https://www.jstor.org/stable/2369451> (visited on 03/05/2023).
- [47] Charles Sanders Peirce. “Prolegomena to an Apology for Pragmaticism”. In: *The Monist* 16.4 (1906). Publisher: Oxford University Press, pp. 492–546. ISSN: 0026-9662. URL: <https://www.jstor.org/stable/27899680> (visited on 10/06/2023).
- [48] Ken Perlin and David Fox. “Pad: An Alternative Approach to the Computer Interface”. In: *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’93. New York, NY, USA: Association for Computing Machinery, Sept. 1993, pp. 57–64. ISBN: 978-0-89791-601-1. DOI: 10.1145/166117.166125. (Visited on 04/14/2025).

- [49] Linda Postniece. “Deep Inference in Bi-intuitionistic Logic”. en. In: *Logic, Language, Information and Computation*. Ed. by Hiroakira Ono, Makoto Kanazawa, and Ruy de Queiroz. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2009, pp. 320–334. ISBN: 978-3-642-02261-6. DOI: 10.1007/978-3-642-02261-6_26.
- [50] Don D. Roberts. *The Existential Graphs of Charles S. Peirce*. Berlin, Boston: De Gruyter Mouton, 1973. ISBN: 9783110226225. DOI: doi:10.1515/9783110226225. URL: <https://doi.org/10.1515/9783110226225>.
- [51] Benoit Rognier and Guillaume Duhamel. “Présentation de la plateforme edukera”. In: *Vingt-septièmes Journées Francophones des Langages Applicatifs (JFLA 2016)*. 2016.
- [52] Sun-Joo Shin. *The Iconic Logic of Peirce’s Graphs*. The MIT Press, May 2002. ISBN: 9780262283557. DOI: 10.7551/mitpress/3633.001.0001. URL: <https://doi.org/10.7551/mitpress/3633.001.0001>.
- [53] Shneiderman. “Direct Manipulation: A Step Beyond Programming Languages”. en. In: *Computer* 16.8 (Aug. 1983), pp. 57–69. ISSN: 0018-9162. DOI: 10.1109/MC.1983.1654471. URL: <http://ieeexplore.ieee.org/document/1654471/> (visited on 01/08/2024).
- [54] Thoralf Skolem. “Logisch-Kombinatorische Untersuchungen Über Die Erfüllbarkeit Oder Bewiesbarkeit Mathematischer Sätze Nebst Einem Theorem Über Dichte Mengen”. In: *Selected Works in Logic*. Ed. by Thoralf Skolem. Universitetsforlaget, 1920.
- [55] John Sowa. “Conceptual Graphs for a Data Base Interface”. In: *IBM Journal of Research and Development* 20 (July 1976), p. 336. DOI: 10.1147/rd.204.0336.
- [56] Lutz Straßburger. “The problem of proof identity, and why computer scientists should care about Hilbert’s 24th problem”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 377.2140 (Mar. 11, 2019), p. 20180038. ISSN: 1364-503X, 1471-2962. DOI: 10.1098/rsta.2018.0038. URL: <https://royalsocietypublishing.org/doi/10.1098/rsta.2018.0038> (visited on 08/08/2020).
- [57] The Rocq Development Team. *The Rocq Prover*. Zenodo. Apr. 2025. DOI: 10.5281/zenodo.15149629.
- [58] Jérôme Vouillon and Vincent Balat. “From bytecode to JavaScript: the Js_of_ocaml compiler”. en. In: *Software: Practice and Experience* 44.8 (Aug. 2014), pp. 951–972. ISSN: 00380644. DOI: 10.1002/spe.2187. URL: <https://onlinelibrary.wiley.com/doi/10.1002/spe.2187> (visited on 01/06/2023).
- [59] Théophile Wallez, Jonathan Protzenko, and Karthikeyan Bhargavan. “Compars: Provably Secure Formats for Cryptographic Protocols”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 564–578. ISBN: 9798400700507. DOI: 10.1145/3576915.3623201. (Visited on 04/11/2025).
- [60] Fernando Zalamea. “Advances in Peircean Mathematics: The Colombian School”. en. In: *Advances in Peircean Mathematics*. De Gruyter, Nov. 2022. ISBN: 978-3-11-071763-1. DOI: 10.1515/9783110717631. URL: <https://www.degruyter.com/document/doi/10.1515/9783110717631/html> (visited on 11/10/2023).